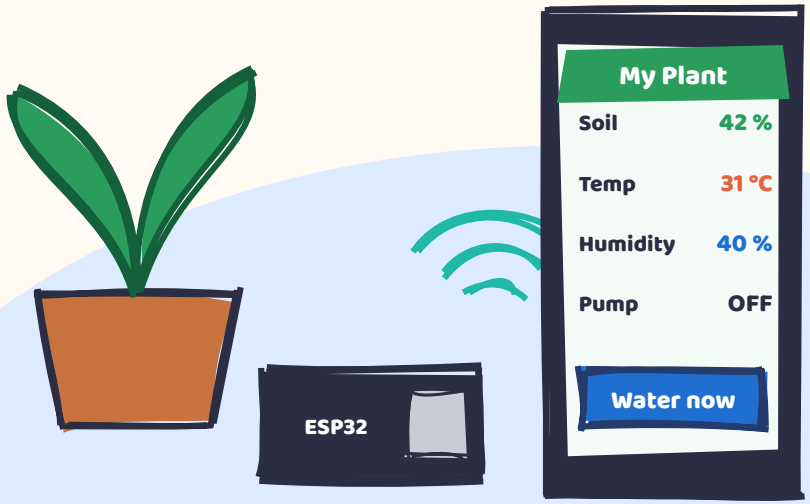
Young Innovators · Class 12

Plant Monitor Mobile App

Check on your plant from your phone!

MIT App Inventor | Web requests | UI design



Did you know?
Almost every smart device you buy, from bulbs to washing machines, comes with its own phone app.

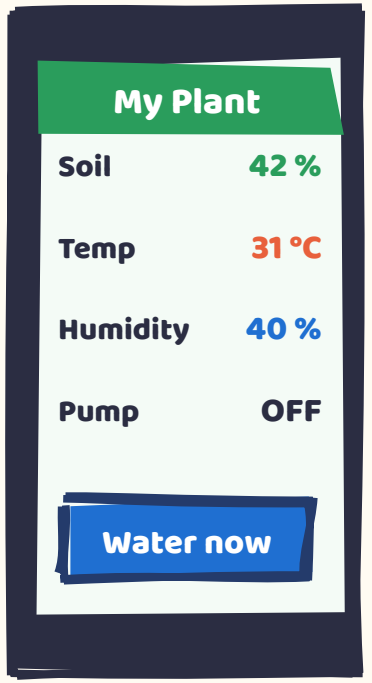
Let's think...

A webpage works. Why build an app?

Our YI-11 plant system

It already shows a webpage. But you must remember the **IP address**, type it in, and press refresh. The page looks small and plain on a phone.

A good app does it all with **one tap**.



Bookmark it?

Write it on paper?

Build our own app!

Today: you design and build an **Android app** that shows live readings and can **water the plant** from your phone.

LEARN

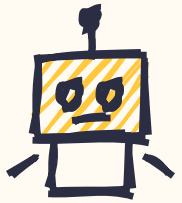
Learning aims

Today I will learn to...

1. Design app screens on **paper first**, for a real user
2. Explain how an app **requests** data from the ESP32
3. Build the app in **MIT App Inventor** with a Web component and a Clock
4. Split text into a **list** and show values with colours and icons
5. **Test** with a classmate and improve the design

Keywords

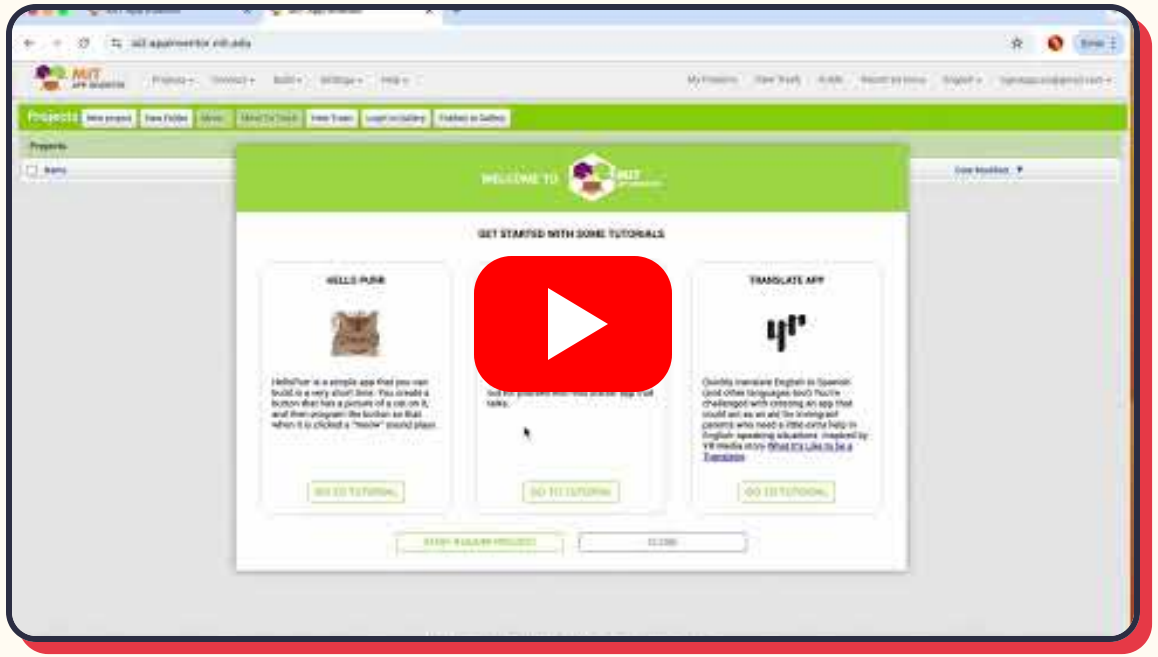
app · UI · request · response · URL · list · event · usability



LEARN

Watch and think

Video time!



MIT App Inventor: Introduction Lesson

twdr · about 3 minutes · youtu.be/k5ayJzrzAx4

Watch for...

1. What are the **Designer** and **Blocks** screens for?
2. How do you test the app on a phone?
3. What kind of app would you build first?

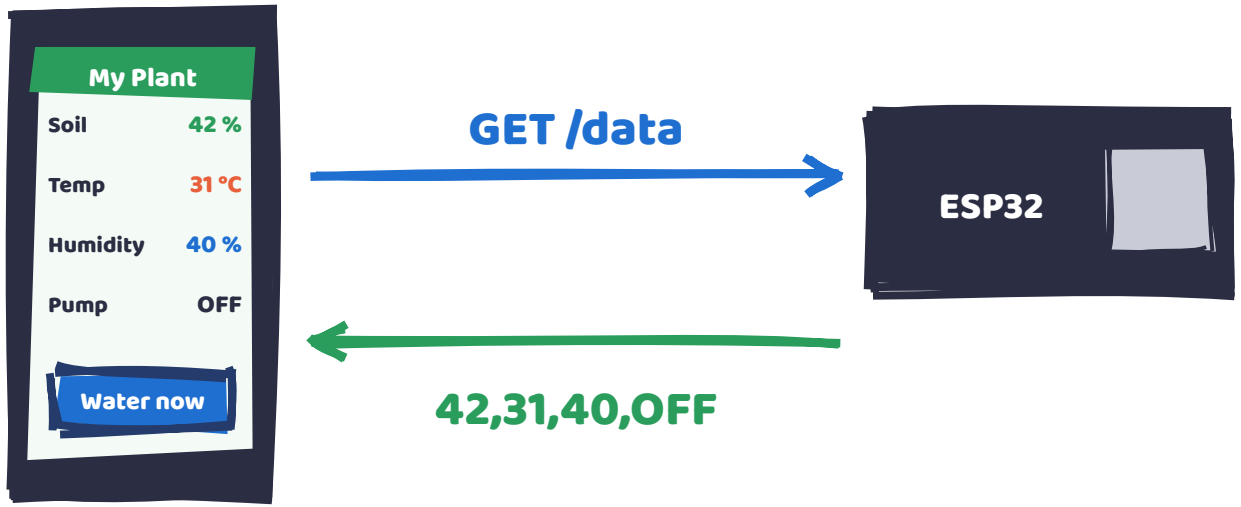
Scan to watch

After the video: Sketch the screen of our plant monitor app: what buttons and labels does it need?

LEARN

Big idea

Ask and answer: request and response



Same as a browser

When you open a website, your browser sends a **GET request** and the server sends a **response**. Our app does exactly the same, every 5 seconds.

Plain text is easiest

Instead of a whole webpage, the ESP32 sends **just the numbers**, separated by commas. The app splits them apart.

URL

http://192.168.1.45/data = the ESP32's address + the page we want.

Real world

Apps that talk to devices



Smart homes
Lights, fans and ACs controlled from one app



Smart farms
Farmers check soil and switch pumps from the field



Fitness bands
Steps and heart rate sent to an app



Cars
Check battery or lock the doors from a phone

? Think of an app you use every day. What makes it EASY to use? What annoys you?

Good design
If users need instructions to use your app, the design needs work.

Part 2 · Make it

Now let's put it to work!

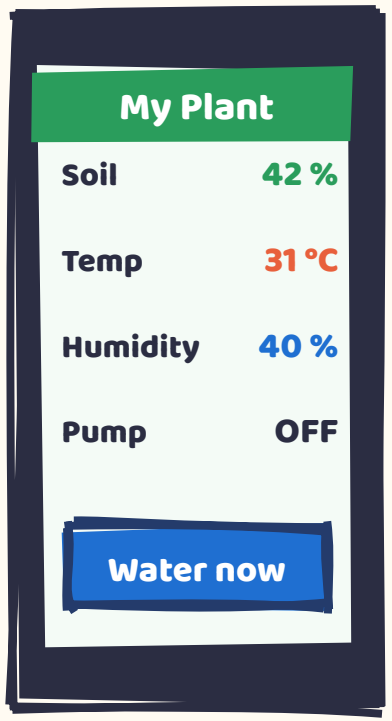
You know the ideas. Time to play, build and test. Use these slides while you work:

1. Sketch your screen on paper
2. Paper prototype test
3. Add two new addresses
4. Check it in a browser before building the app
5. The Designer screen
6. Ask for new data every 5 seconds
7. Split the reply and show it

MAKE

Design first

Sketch your screen on paper



Most important on top

What does the user need to know **first**? Probably: does my plant need water?

Big touch targets

Buttons at least the size of a **fingertip** (about 1 cm).

Colour has meaning

Green = fine, **red** = needs attention. Add words too, for colour-blind users.

Your user

Design for a real person: a **busy parent**, a **grandparent** with big text settings, or a **gardener** outside in sunlight.

Template

Use the printed phone-screen sheet. Pencil only, so you can change it!

MAKE

Class game

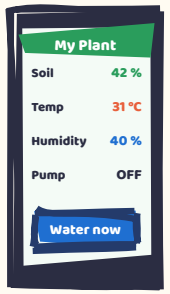
Paper prototype test

How to play

1. Swap your paper sketch with another team.
2. One person plays the **user**: 'tap' the paper with a finger.
3. The designer plays the **computer** and slides in new paper screens.
4. Task: **find out if the plant needs water, then water it**.
5. Note every place the user hesitated.

Then discuss

Where did the user get stuck? What would you change BEFORE writing any code?



Big idea

Fixing a design on paper takes 1 minute. Fixing it in code can take an hour.

MAKE

Step 1: the ESP32

Add two new addresses

```
// Add to your YI-11 ESP32 code, in setup():
server.on("/data", []() {
  String s = String(soilPct) + "," + String(temp, 1) + ","
           + String(hum, 0) + "," + (pumpOn ? "ON" : "OFF");
  server.send(200, "text/plain", s);
});

server.on("/water", []() {
  runPump(3000);                          // pump ON for 3 s
  server.send(200, "text/plain", "OK");
});
```

/data

Replies with one line of text, for example **42,31.0,40,OFF**.

/water

Runs the pump for 3 seconds, then replies **OK**.

Safety limit

Make runPump() refuse to run again within 1 minute, so nobody floods the plant by tapping 20 times!

Step 2: test first

Check it in a browser before building the app

1. Upload the new ESP32 code. Note the **IP address** from the Serial Monitor.
2. Connect your phone to the **same Wi-Fi**.
3. Open **http://YOUR-IP/data** in the phone's browser. You should see something like **42,31.0,40,OFF**.
4. Open **/water**. Does the pump run for 3 seconds?

School Wi-Fi blocks it?

Some networks stop devices talking to each other. Use the teacher's **phone hotspot** or a separate router instead.

Engineer habit

Test each part on its own. If the browser fails, the app will fail too!

MAKE

Step 3: App Inventor

The Designer screen

Component	Used for
Label × 4	soil, temperature, humidity, pump
Button	Water now
Image	happy / thirsty plant icon
Web (×2)	one for /data, one for /water
Clock	TimerInterval = 5000 ms
Notifier	pop-up warnings

Where?

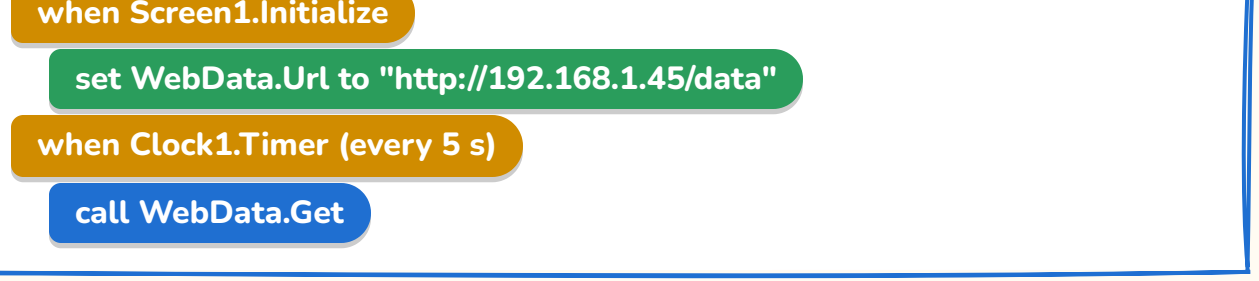
Open **ai2.appinventor.mit.edu**, start a new project, and drag components from the Palette. Hidden ones (Web, Clock, Notifier) appear under the screen.

Live testing

Install **MIT AI2 Companion** on the phone and connect with the code. Changes appear instantly!

Step 4: blocks

Ask for new data every 5 seconds



Why 5 seconds?

Fast enough to feel **live**, slow enough not to overload the ESP32 or drain the phone battery.

Events

App Inventor programs are made of **events**: when the screen opens, when the timer ticks, when a button is clicked, when data arrives.

Use YOUR IP

Every ESP32 gets its own address. Copy it from the Serial Monitor.

MAKE

Step 5: blocks

Split the reply and show it

```
when WebData.GotText (responseContent)
    set parts to split responseContent at ","
    set SoilLabel.Text to join "Soil: " (select list item parts, index 1) " %"
    set TempLabel.Text to join "Temp: " (select list item parts, index 2) " °C"
    set HumLabel.Text to join "Humidity: " (index 3) " %"
    set PumpLabel.Text to join "Pump: " (index 4)
```

index	value
1	42
2	31.0
3	40
4	OFF

Index starts at 1!

In App Inventor the first list item is **index 1**. (In Python and C++ it is 0!) A common bug.

42,31.0,40,OFF
split at comma makes a list of 4 items.

Step 6: control

The 'Water now' button

```
when WaterButton.Click
    call Notifier1.ShowChooseDialog "Water the plant now?" Yes / No
when Notifier1.AfterChoosing (choice)
    if choice = "Yes"
        set WebWater.Url to "http://192.168.1.45/water"
        call WebWater.Get
        call Notifier1.ShowAlert "Watering for 3 seconds"
```

Ask before acting

A confirm pop-up stops **accidental taps** from flooding the plant. Good apps confirm actions that change the real world.

Feedback

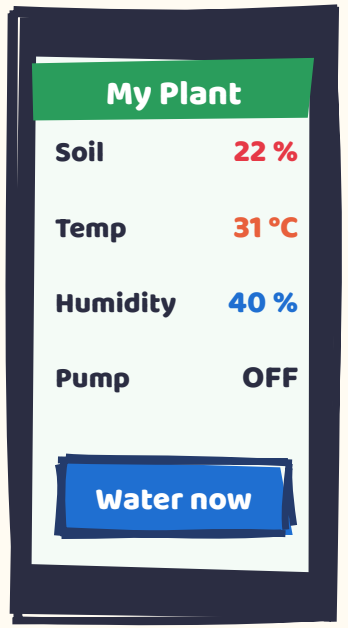
Always tell the user something happened. Silence feels broken!

MAKE

Step 7: make it clear

A low-water warning

```
if (select list item parts, index 1) < 30
    set SoilLabel.TextColor to red
    set PlantImage.Picture to "thirsty.png"
else
    set SoilLabel.TextColor to green
    set PlantImage.Picture to "happy.png"
```



Your threshold

Use the **DRY** level you measured in YI-11, converted to %. Don't guess!

Text AND colour

Write 'Needs water!' too. About 1 in 12 boys has some colour blindness.

Test like an engineer

Test with a real user

Task for the user	Time	Without help?
Open the app and say if the plant needs water	____ s	yes / no
Water the plant	____ s	yes / no
Say the temperature	____ s	yes / no
Find what to do if it shows an error	____ s	yes / no

Debug checklist

App shows nothing? Test /data in the browser again.
Values in the wrong labels? Check index numbers (start at 1).
Error 1101? Phone and ESP32 on different Wi-Fi.

Stay silent while they test! Only watch and take notes. Then improve ONE thing.

MAKE

Level up!

App challenges

Choose your challenge



History: show the last 5 soil readings as a list, or draw a tiny chart on a Canvas.



Alert: vibrate the phone when soil drops below the threshold.



Settings screen: let the user change the IP address without editing blocks.



Plant care tips: show a different tip based on temperature and humidity.

Think like an engineer

Right now the app only works on the same Wi-Fi. How do real smart-home apps work from anywhere in the world?

Boss level

Devices send data to a cloud server, and the app reads from the cloud. Tech Leaders build this!

Show what you know

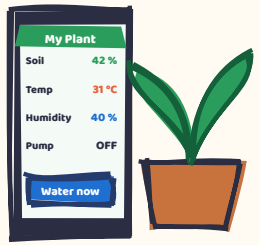
What did we learn?

- [x] Design on paper first, for a real user
- [x] The app sends a GET request; the ESP32 sends a response
- [x] Plain text like 42,31.0,40,OFF is split into a list
- [x] App Inventor list indexes start at 1
- [x] Confirm actions and give clear feedback

Exit ticket: why did we add a confirm pop-up to the 'Water now' button?

At home

Show your app to a family member. Can they use it without your help?



Next class: a Python Maths Quiz Game!